S2 Text. Task prompts given to the agent

1. Original task prompt (verbatim)

This is the prompt that started the project (copied verbatim from `PROMPT.md` in the research repository).

> /goal You are the Principal Investigator that serves as the scientific lead of a Comparative genomics project and work autonomously. You lead a team of agents (Opus 5.5 or Sonnet 5.5, as you see fit). You delegate, review, integrate and decide; you keep the research record. The goal is the best possible outcome, not the fastest completion. Make decisions yourself, document why, and keep going. Do not stop until you obtain a new orthology prediction method that is conceptually different to the existing, state-of-art methods, but that has a comparable if not better overall benchmark inference orthology and/or computation time, and with all of the desired characteristics explained in the rest of the prompt that are doable within the allocated time.
>
> Background: Homologous genes are genes that share a common evolutionary origin because they descended from a single ancestral gene. Similarity between gene sequences implies common ancestry. Genes which are related through speciation are called orthologs, as opposed to paralogs, which are related through duplication. Methods such as OrthoFinder or OMA use all-against-all sequence comparisons (implemented in BLAST or Diamond). This computation is O(N^2), which is slow.
>
> Task: Build an orthology prediction method that is applicable to a scalable number of proteomes (from few dozens to multiple thousands; potentially, up to alphafold database scale) and that reports the inferred relationship of the proteins into hierarchical orthologous groups. The output should include at least a file in the orthoxml format. A successful method would assign all the genes from all given genomes in the correct evolutionary relationships between them, identifying the genes that are all descending from one ancestral gene.
>
> You have complete freedom in the approach to use for the whole task, but you will need to split the problem into smaller, improvable sub-tasks, each with its own benchmark and quality control. For example, in case of a tree-based approach, the tree-building step should be improved and benchmarked by its own before the full pipeline benchmark; similarly, in case of an initial protein family placement step, the protein family placement must be benchmarked itself and optimised, before making it inside the pipeline final benchmark. Another critical point will be the usage or computation of a species tree. Explore the alternatives regarding using a universal species tree, since several protein families might have alternative evolutionary histories that will be difficult to model univocally.
>
> The method will likely use protein sequence information as this is the predominant form of data that is accessible on online databases for genes. However, test the possibility of including additional data available to support the orthology prediction task. Here you can see a non-complete list of the data you can try to add in any of the sub-steps required to complete the main task. If possible, report the effect of using or not using the additional layers of data in the final benchmark as well as in the sub-task specific benchmark:
>
> Protein 3d structure; the recent scientific development stemming from Alphafold3 and several protein language models resulted in an abundance of predicted protein structure or protein structure representation across the whole tree of life and for several entire genomes. This information could be beneficial for remote homology (foldseek), tree building (foldtree), and thus, eventually, orthology. Try to integrate protein 3d structure in orthology prediction in any way it seems fit. One of the initial attempts could involve the 3di representation of protein structure, because it neatly corresponds to

protein sequence, is easily obtainable at scale, and has been integrated in several existing methods.

Synteny; the genomic context tends to be maintained over some evolutionary times. So far this has not been exploited very much in orthology prediction methods, but rather in some post-processing steps as verifications. There could be potential to integrate this information in the predicting part of orthology as well. For most reference genomes, the .gff file containing gene location is obtainable along the genome and proteome from the reference database.

Genomic context; multiple additional layers of data exists, mostly from OMICs experiments on model species, that could be explored as a way to detect conservation, for example tracking the corresponding copy of paralogous genes using gene expression, epigenetics, etc. Explore the possibility of including protein expression, chip seq or methylation experiments, or other ancillary annotations that could help the orthology inference. Keep track and report the conservation and conservation breaks of this information. This data is not trivial to obtain at a large scale, therefore feel free to discard this idea if it is not feasible for implementing in the overall task. However, make an attempt to test if it could help in any way.

Isoforms; When available, you could use isoform information as an additional layer of information. Alternative splicing might only be useful at certain taxonomic ranges and only a few representative lineages. However, it might contain interesting information in addition to establish orthology, or even orthologous isoforms.

Genome (DNA) sequence; it might be re-used to validate the effective loss of a gene. For example, when an orthologous group is not found in specific taxa, the genomic locus of neighbouring orthologous genes might be investigated to validate the effective loss, by observing changes in local synteny or pseudogenization. This might give additional data to validate gene presence/absence and the placement of the gene orthologous group in the correct taxonomic level.

Finally, to validate the method, use all of the existing orthology benchmarks developed by the community, like QfO benchmark (https://github.com/qfo/benchmark-webservice), orthobench (https://github.com/davidemms/Open_Orthobench), swisstree (https://swisstree.vital-it.ch/cgi-bin/swisst), etc. Compare to other existing methods in terms of accuracy and scalability. But keep the benchmark of the method in a separated repository that could be easily reproduced by a user that wants to validate the method step by step.

If you cannot easily Do not waste time trying to access datasets or other materials; ask the user but don't wait, continue with other work instead. Flag and explain all important decisions clearly. You have a time limit given to the cluster accounting of the job. This time limit is 48 hours; Have the harness write START.txt at launch and tell the agent to check date against it at each milestone. plan ahead your experiments accordingly, you can do quick tests on how long things take to run on the provided hardware before committing a long run. Do not be scared of doing long runs but just plan regular milestones so that in case the run could be resumed if suddenly interrupted.

You will be running this project in a singularity container inside a node job using slurm, in our university cluster, in the scratch folder. Keep a git repository that documents everything and that later will be put together in github with all other projects of this hackathon and accessed by other agents to have new phase2 projects that will use the method and results stemming from this project. You will work entirely on my user /scratch folder, where data is stored temporarily but you do not have constraints of file size and number. At the end of the project, I will manually connect a remote for the git repository and save all the relevant code and the minimal data that we need to test and verify later. You will use an interactive job with 24 cpus and 150 gb of memory. Consider the possibility of gpu optimization of the code, however you current allocation does not include any gpu resources.

Literature: Design through repeated and independent critique: Search the current literature and available datasets, and write LITERATURE.md. Then draft DESIGN.md. The project goal and its parts are fixed, the suggestive ideas are a human priors, not a plan. Have the design reviewed and revised in at most three rounds by an external agentic PI, and agentic scientists assessing prior work, statistics, data, algorithms and compute, AI-agent failure modes, and other expertise the design needs. Each reviewer proposes concrete changes that stay within the project's scope, or states that none are

needed.

Reproducibility: Never change metrics, controls, labels or tests to make results look better. Keep everything end-to-end reproducible: commit results to git, fix the random seeds list, and keep DATA.md listing the source, version, checksum and license of each dataset. Never commit restricted data, never hardcode data, prefer scripts over one-offs, make sources explicit, no silent assumptions. Don't trust a result until it has been checked. Write verification scripts using toy data with known answers, positive and negative controls, and sanity checks. An agent that didn't produce a result reviews it before you rely on it.

Research tree: Save this prompt verbatim to PROMPT.md. Keep track of the research in a research tree (RESEARCH_TREE.md), using a status legend: [ ] planned · [~] in progress · [x] done · [-] abandoned/negative · [!] key finding. At each milestone, add a short decision note and a two-paragraph status summary at the top. The tree is your memory: keep your own context lean by reading agents' summaries by default rather than raw logs and if your context is reset, resume from the tree.

AI lessons: Keep LESSONS.md, a list of general process rules for running this project efficiently. Whenever something wastes or saves significant effort, add a one-line rule with its evidence. At each milestone, prune it to the most useful rules and ask a fresh agent "what should we stop doing?" and "what worked well?".

Auto-research: Your job is to build the best possible method. Report runtime alongside accuracy. Keep going until you genuinely run out of promising ideas or close to the time limit. Then write REPORT.md with a breakdown of the approach, the best methods, test-set results, follow-ups, limitations, and remaining ideas.

2. Later human instructions (summarised; verbatim where quoted)

1. Suggestion to consider the Foldcomp prebuilt structure databases (https://github.com/steineggerlab/foldcomp).
2. "include at least one OMA resource in all benchmark/comparisons from now on, for example fastOMA".
3. Pause all processes safely, commit, write a resume file and a one-page report, and move the project to a GPU node (48 CPUs, 500 GB RAM, one A100) to explore GPU speed-ups.
4. Test GPU access and resume from the original prompt and the resume document.
5. Keep the original 48-hour budget, now as a soft deadline; add a fixed human-written header to the top of both repository READMEs.
6. Write this manuscript, a supplement of failed methods and a minimal public repository (prompt in section 3).

3. Manuscript prompt (verbatim)

You remain the PI of this computational project. Now your goal is to write the results of the project into a manuscript that explains the winning method and presents your benchmarking of it, formatted for the journal PLOS Computational Biology. This should be accompanied by an mvp github repository that will include the predictions and will allow readers to implement the method themselves (set up the repo and allow me to publish it myself). In both the paper and the repo, you should explicitly flag at the beginning that this predictive task was an activity in testing the ability of agents to run bioinformatics projects; the repo should include the original prompt, and the methods section should explain the agent and sub-agent setup. For the final method that worked, you should give credit to the ideas that you extracted from the literature. Write a supplementary file detailing the major methods you tried that did not work. In the paper, explain the method both conceptually and technically. Use this skill for writing https://github.com/jurgjn/euler-vibe/blob/main/home/euler-claude/.claude/skills/edit-for-clarity/SKILL.md. Mention and explain all important decisions you took and clearly explain your approach. Be critical and do not overstate. Stay precise, keep it short and to the point when possible.